Java: E-Commerce Cart Manager

Implement a Java program to calculate the total checkout cost of an e-commerce shopping cart. The program should:

- Support percentage and flat rate discount coupons by extending a base class
- Apply a 10% tax rate ONLY to items marked as taxable
- Handle invalid coupon configurations (e.g. empty coupon code or invalid discount values) using a custom checked exception

Class Description

Complete the classes `PercentageDiscount` and `FlatDiscount` which extend the abstract class `DiscountCoupon`. You must also implement the class `CartService`.

1. **Item (Pre-provided)**:
2. `String name`: Stores the product name.
3. `double price`: Stores unit price.
4. `int quantity`: Stores number of units.
5. `boolean isTaxable`: Indicates if the item is taxable.
6. **DiscountCoupon (Pre-provided abstract class)**:
7. `String code`: Stores the coupon code.
8. `abstract double getDiscount(double subtotal)`: Abstract method to compute discount.
9. **PercentageDiscount**:
10. Constructor `PercentageDiscount(String code, double percentage)`
11. The discount amount is calculated as: `subtotal * (percentage / 100)`.
12. **FlatDiscount**:
13. Constructor `FlatDiscount(String code, double amount)`
14. The discount amount is simply `amount`. It must not exceed the subtotal (if it does, the discount is capped at the subtotal, resulting in a discounted subtotal of 0).
15. **InvalidCouponException (Pre-provided exception)**:
16. Checked exception thrown during coupon validation.
17. **CartService**:

    `double calculateTotal(List<Item> items, DiscountCoupon coupon) throws InvalidCouponException`:

    - Validation Rules:
    - If `coupon` is not null, and its code is null or empty (after trimming whitespace), throw `InvalidCouponException("Coupon code cannot be empty.")`.
    - If `coupon` is a `PercentageDiscount`, the percentage must be between 0 and 100 inclusive. Otherwise, throw `InvalidCouponException("Invalid discount value.")`.
    - If `coupon` is a `FlatDiscount`, the amount must be non-negative. Otherwise, throw `InvalidCouponException("Invalid discount value.")`.
    - Total Calculation:
    - Calculate the subtotal as the sum of `price * quantity` for all items.

- If the subtotal is 0 (or list is empty), the final total is `0.0` (no exception is thrown even if the coupon value is invalid, as checkout total is immediately 0).
- Compute the discount using the coupon's `getDiscount(subtotal)` method.
- Calculate a 10% sales tax (0.10) applied *only* to the subtotal of items where `isTaxable` is `true` (calculated on pre-discount price).
- The final total is computed as: `subtotal - discount + tax`.

Returns * `double`: The final cart checkout total.

Constraints * $0 \le NumberOfItems \le 100$. * $0.0 \le price \le 10,000.0$. * $0 \le quantity \le 50$.

Input Format For Custom Testing * The first line contains an integer N representing the number of items. * The next N lines each contain: `{Name} {Price} {Quantity} {IsTaxable}`. * The final line contains: `{CouponType} {CouponCode} {CouponValue}`. If no coupon is applied, this line contains `None` or has placeholder values.

Sample Case 0 Sample Input For Custom Testing

```
3
Laptop 1000.00 1 true
Mouse 50.00 2 false
Notebook 5.00 5 true
Percentage SAVE10 10.0
```

Sample Output

```
Subtotal: 1125.00
Discount: 112.50
Tax: 102.50
Total: 1115.00
```

Explanation * Subtotal = (1000.00 * 1) + (50.00 * 2) + (5.00 * 5) = 1125.00 * Discount (10% of 1125) = 112.50 * Taxable items subtotal = Laptop (1000.00) + Notebook (25.00) = 1025.00. Tax (10%) = 102.50. * Total = 1125.00 - 112.50 + 102.50 = 1115.00.

Sample Case 1 Sample Input For Custom Testing

```
1
Phone 500.00 1 true
Percentage SAVE200 150.0
```

Sample Output

```
Error: Invalid discount value.
```

Explanation * The discount percentage is 150.0%, which is invalid (> 100%).

Boilerplate:

```java
// student_workspace/Solution.java

import java.util.*;

/*
* Implement the 'PercentageDiscount', 'FlatDiscount', and 'CartService' classes here.
* The 'Item', 'DiscountCoupon' and 'InvalidCouponException' classes are already defined in the Harness.
* DO NOT use access modifiers (e.g.: 'public') in your class declarations.
*/

class PercentageDiscount extends DiscountCoupon {
   // Write your code here
}

class FlatDiscount extends DiscountCoupon {
   // Write your code here
}

class CartService {
   // Write your code here
}
```